

Homework 1

Integer and Floating Point Number Representations

Integer

Problem 1

Suppose you have a 2 GHz x64 core and you can execute three integer operations (additions, subtractions, comparisons) every cycle.

1. How long (how many seconds) will the following loop run?

```
int32_t i;           /* 32 bit signed integer */
uint64_t s = -1;     /* 64 bit unsigned integer */
int16_t z = 0;       /* 16 bit signed integer */
for (i = 0 ; i >= 0 ; i++) {
    s++;
    z++;
}
```

2. How many times does s overflow?
3. How many times does z overflow?

Problem 2

How can you compute the following using only shifts, adds, and subtracts? Here, x is a uint32_t.

```
8 * x
14 * x
37 * x
x / 16
x / 5   (Hard! Outside the scope of class, but included as a challenge)
```

Problem 3

Consider a 3 bit unsigned integer. Consecutive values can change by several bits. For example in going from 011 (3) to 100 (4) all 3 bits change. In some cases, this can take enough energy to be a problem. Create an encoding for 3 bit unsigned integers that has the property that consecutive numbers always change by only a single bit. This includes the wrap-around from 7 to 0. Show a table.

Problem 4

Many instruction sets have instructions where when you multiply two k bit numbers the result is stored as two k bit numbers. Why? Similarly, many have instructions where if you divide two k bit numbers, the result is stored as two k bit numbers. Why? By “k bit numbers” think of unsigned integers, although the same reasoning holds for signed integers.

Problem 5

Consider the following code. Determine the values of c, d, e, and f. Show your work.

```
int main(void)
{
    short a = 15;
    short b = 3858;
    short c;
```

```

short d;
short e;
short f;

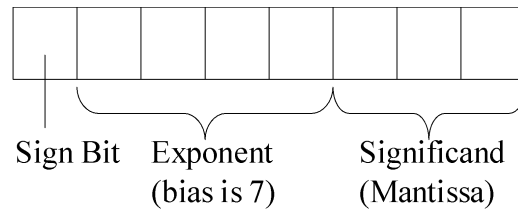
c = a | 16;
d = b ^ c;
e = ~d && (b & c);
f = e && !e;
}

```

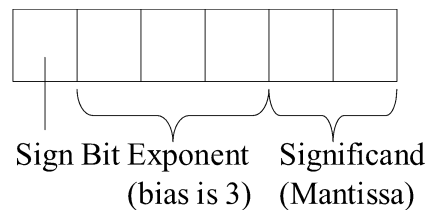
Floating Point

Consider the following two small floating point formats based on the IEEE standard:

- Little Format



- Tiny Format



Except for the sizes of these formats, the rules are those of the IEEE standard.

Problem 1

For both formats, determine the following values (in decimal)

1. Largest positive finite number
2. Positive normalized number closest to zero
3. Largest positive denormalized number
4. Negative denormalized number closest to zero

Problem 2

Encode the following values in the 8 bit Little Format or report that they cannot be so encoded: $7/4$, $-1/16$, 144, -35, NaN, and infinity. Show each in binary and hexadecimal, if possible.

Problem 3

Determine the values corresponding to the following Little Format bit patterns. The leftmost bit is the most significant

1. 01010010
2. 00000111
3. 01001010
4. 10010000
5. 00111010
6. 11111001

Problem 4

Convert the following 8 bit Little Format numbers into 6 bit Tiny Format numbers. Overflow should yield +/- infinity, underflow should yield +/- 0.0, and rounding should follow the “round-to-nearest-even” tie-breaking rule. Show the bit pattern and its hex representation.

1. 01111001
2. 11111000
3. 01001100
4. 00110011
5. 11001010
6. 01111000
7. 01111111

Problem 5

In machine learning, especially inference, smaller floating point numbers are considered acceptable.

- A 16 bit version of the IEEE standard exists and is widely implemented. A float16 has 5 exponent bits and 10 mantissa bits.
- Brain Floating Point is another important standard (also widely implemented in recent Intel and ARM chips, and originating in Google’s TPU chips). A bfloat16 has 8 exponent bits and 7 mantissa bits. It is essentially a 32 bit IEEE floating point number with 16 bits chopped off the end of the mantissa.
- Two 8 bit versions of IEEE (float8) exist and are increasingly implemented in hardware. E4M3 has a 4 bit exponent and a 3 bit mantissa (just like the “Little” format from above) , while E5M2 has a 5 bit exponent and a 2 bit mantissa.
- 4 bit floats (float4) exist in NVIDIA GPUs. They have 2 exponent bits and 1 mantissa bit (E2M1).

What are the comparative advantages and disadvantages of these formats in terms of the real numbers they can represent? Think in terms of precision and range.

For float8, why choose E4M3 versus E5M2?

How many addition problems of the form $x + y$ exist, where x and y are float4s?

Problem 6

The floating point (and integer, fixed point, bcd, etc) numbers and their operators (+/*/...) are generally implemented in hardware for speed. However, software implementations also exist, such as GNU MPFR. While these are much slower, they allow a programmer to use a bit width that the hardware does not support by emulating it using software. For example, the programmer could choose to use 1024 bit floating point numbers or 2048 bit integers. Why might a programmer do this?

(Thought experiment – nothing to hand in) Could you write a program that implements a rational number system? In a rational number system, every value is represented as a fraction (a ratio of two integers).

Other

Problem 1

1. In the textbook (Section 2.1.4) you are introduced to C-style strings and the `show_bytes` function (Section 2.1.3, Fig 2.4). **What would be printed as a result of the code snippet below?**

```
const char *s = "tonight's the night.";
show_bytes( (byte_pointer) s, strlen(s) + 1 );
```

2. Alternative string representations store relevant metadata associated with raw pointer to the string data, to make up for the short-comings of C-style strings. Consider this:

```
typedef struct mystring {
    uint64_t length; // length of string
    uint64_t hash;   // hash of string content
    const char *s;   // string data
} mystring_t;
```

How would you write a function that checks if two `mystring_ts` are the same?

Would the length and hash metadata help you in checking if one string is less than another? For example, when comparing “Smith” and “Jones”, “Jones” would be less than “Smith” because “Jones” would appear first in lexical order.

3. Early computers used Baudot Codes from the 19th century to encode characters. These allowed 5 bits per character, so 32 symbols could be encoded. 2 of those symbols meant “switch to letters mode” and “switch to figures mode”. The mode determined how the other 30 symbols were interpreted as characters. How many possible characters could be represented? Is that enough?

New computers use UTF-8, 16, 32 character encodings – the number corresponds to the bitwidth used to encode characters. Why are characters up to 32 bits wide useful?